

SmartGear Retail 2.0

End-to-End Data Platform Architecture

Architecture Diagram & Design Document

Assignment: SmartGear Retail 2.0
Submitted By: Rishabh Srivastava

Executive Summary

This document presents the comprehensive end-to-end data platform architecture for SmartGear Retail 2.0, designed to handle batch, streaming, and semantic data sources. The platform leverages modern data engineering best practices including the Medallion Architecture (Bronze-Silver-Gold layers) with Delta Lake for ACID transactions and reliable streaming pipelines.

Platform Architecture Layers

Layer	Component	Technology	Purpose
Data Sources	Batch Dataset	CSV Files (S3)	Historical sales data from Assignment1
	Real-time Orders	POS Systems	Live order streams from retail locations
	Product Metadata	Product Database	Catalog and descriptions for semantic search
Ingestion	Cloud Storage	S3/ADLS	Batch file landing zone
	Kafka Topic	Confluent Cloud	smartgear_orders topic with 7-field schema
	Product DB	Source database	Metadata for embedding generation
Processing	Spark Batch	Databricks	Historical data processing
	Spark Streaming	Databricks	Real-time order processing
	Embeddings	Databricks	Vector generation for semantic search
Storage	Bronze Layer	Delta Lake	Raw, append-only data with metadata
	Silver Layer	Delta Lake	Cleaned, deduplicated, standardized data
	Gold Layer	Delta Lake	Business KPIs and executive-ready analytics
Semantic	Vector DB	Chroma	Embeddings storage and similarity search

Analytics	Synapse	Azure Synapse	SQL pools for complex queries
	BI & Dashboards	Power BI/Databricks SQL	Executive reports and dashboards
	Semantic Apps	Custom Apps	Product search and recommendations
Orchestration	DAG Orchestration	Airflow/ADF	Pipeline scheduling and monitoring

Data Flow & Processing Pipeline

1. Ingestion Phase:

- Batch data from CSV files lands in S3/Cloud Storage
- Real-time orders stream via Kafka to Confluent Cloud topic "smartgear_orders"
- Product metadata continuously available from source database

2. Processing Phase:

- Spark Batch jobs process historical sales data
- Spark Structured Streaming consumes Kafka messages in micro-batches
- Schema enforcement ensures data quality during ingestion
- Embeddings generated from product descriptions for semantic search

3. Medallion Architecture:

Bronze Layer (Raw): Append-only Delta tables preserve raw data with ingestion_time and source metadata

Silver Layer (Cleaned): Data engineering transformations including null handling, standardization, deduplication, and revenue calculation

Gold Layer (Analytics): Executive-ready KPIs including daily trends, regional contributions, store rankings, anomaly detection, and performance segmentation

4. Analytics & Consumption:

- Azure Synapse analyzes Gold layer data via SQL pools
- Power BI/Databricks SQL creates executive dashboards
- Vector Database stores embeddings for semantic search functionality
- Semantic search API enables product discovery and recommendations

5. Orchestration:

- Airflow DAGs or Azure Data Factory orchestrates end-to-end pipeline
- Kafka → Bronze → Silver → Gold workflows scheduled with monitoring
- Cost optimization and performance tracking built-in

Key Design Decisions & Justifications

Lakehouse Architecture Over Traditional Data Warehouse:

Delta Lake provides ACID transactions, schema enforcement, and time-travel capabilities while supporting both batch and streaming workloads. This eliminates data silos and reduces complexity compared to separate operational and analytical systems.

Medallion Architecture (Bronze-Silver-Gold):

This three-layer approach ensures data quality, reproducibility, and auditability. Each layer serves a specific purpose: raw ingestion, transformation, and analytics-ready output. Enables easy backfills and error recovery.

Confluent Cloud for Streaming Ingestion:

Kafka provides high-throughput, fault-tolerant message streaming with topic partitioning for scalability. Confluent Cloud managed service eliminates operational overhead compared to self-managed Kafka clusters.

Spark Structured Streaming for Real-time Processing:

Provides exactly-once semantics with checkpointing, enabling reliable streaming pipelines. DataFrame API integrates seamlessly with Databricks ecosystem for batch-streaming unification.

Delta Lake's Time-Travel Feature:

Enables auditing, debugging, and recovery from data quality issues. Critical for streaming pipelines where late-arriving or out-of-order data must be managed gracefully.

Azure Synapse for Analytics:

SQL Pools provide MPP (massively parallel processing) for complex analytical queries. Integration with Power BI enables self-service BI while maintaining data governance.

Vector Database for Semantic Search:

Embeddings enable similarity-based product search beyond keyword matching. Chroma provides lightweight local storage and ANN (Approximate Nearest Neighbor) for efficient similarity lookups.

Data Quality & Governance

Schema Enforcement:

Delta Lake enforces schema at the Bronze layer. Changes flow through Silver via schema evolution while maintaining backwards compatibility.

Deduplication:

Silver layer removes duplicates based on order_id + timestamp, ensuring one fact per event.

Null Handling:

Strategy documented in code with justification (drop vs. impute). Nulls handled consistently across all transformations.

Data Validation:

- Region values standardized via mapping tables
- Price and quantity fields validated for positive values
- Timestamp validation ensures chronological ordering

Monitoring & Alerting:

- Row counts tracked at each layer for reconciliation
- Kafka lag monitored to detect processing delays
- Gold layer metrics compared against batch outputs for validation

Performance & Cost Considerations

Streaming Latency vs. Cost:

Micro-batch processing (vs. true streaming) reduces costs by batching 1000s of records at once. End-to-end latency: sub-second for Kafka ingestion + seconds for processing = low-latency analytics (not real-time).

Storage Optimization:

- Partitioned by date in Silver/Gold layers for efficient queries
- Compaction scheduled off-peak to consolidate small files
- Retention policies: raw data kept 90 days, aggregates indefinitely

Compute Efficiency:

- Auto-scaling clusters adjust worker count based on workload
- Streaming clusters smaller than batch to save costs
- Spot instances used for non-critical workloads

Query Optimization:

- Materialized views in Gold layer pre-compute expensive aggregations
- Partitions and indexing on region/store/date for fast filtering
- Caching of frequently accessed datasets

Cost Trade-offs:

- Kafka: managed service cost vs. operational simplicity
- Databricks: higher per-DBU cost justified by operational simplicity and ecosystem integration
- Synapse: complex queries justify SQL Pool elasticity; simple queries use serverless

Failure Scenarios & Recovery Mechanisms

Scenario: Kafka Lag Spike

Root Cause: Processing job stalled or underprovisioned workers

Detection: Kafka Consumer Group lag metric exceeds threshold

Recovery: Auto-scale Spark cluster, resume processing from checkpoint (no data loss)

Scenario: Data Quality Issue in Silver

Root Cause: Corrupted data or failed transformation logic

Detection: Reconciliation between streaming and batch Gold outputs reveals mismatch

Recovery: Use Delta time-travel to revert Silver table, fix code, replay from checkpoint

Scenario: Missing Records in Gold Layer

Root Cause: Filter condition incorrectly dropped valid rows, or late-arriving data

Detection: Row count audit queries show fewer records than expected

Recovery: Examine Kafka offset logs, replay missing time window through pipeline

Scenario: Streaming-Batch Revenue Mismatch

Root Cause: Different aggregation logic, time-zone misalignment, or late data not handled

Detection: Compare daily totals: GOLD (streaming) vs. BATCH (historical)

Recovery: Implement late data handling window (30-60 min), regenerate affected dates

Technical Specifications & Configuration

Component	Configuration	Details
Kafka Topic	smartgear_orders	Partitions: 6 (by store_id), Retention: 7 days, Replication: 3
Bronze Layer	Append-only mode	Schema: order_id, timestamp, store_id, product, quantity, price, region
Silver Layer	Merge mode	Deduplication on (order_id, timestamp), revenue = quantity * price
Gold Layer	Overwrite mode	Daily partitions, materialized views for KPIs
Databricks Cluster	Streaming	2-4 workers, 8GB memory, auto-scaling enabled
Databricks Cluster	Batch	4-8 workers, 16GB memory, scheduled jobs
Vector DB	Chroma	Embedded mode, 384-dim embeddings (OpenAI/HuggingFace)
Synapse Pool	DW500c	Scalable for complex aggregations and historical analysis
Checkpointing	Distributed	S3/ADLS location with 30-day retention

Service Level Agreements (SLAs) & Performance Targets

Streaming Latency: End-to-end from Kafka to Gold $\leq$ 5 minutes (99th percentile)

Batch Processing: Daily historical load completes within 2 hours

Data Freshness: Gold layer refreshed every 10 minutes for streaming, daily for batch reconciliation

Availability: Pipeline uptime target 99.5% (rolling 30-day window)

Data Accuracy: Streaming-batch reconciliation within 0.1% variance allowed

Query Performance: 95th percentile query latency for Gold analytics $\leq$ 30 seconds

Cost Target: Cost per million events processed $\leq$ \$5 USD

Future Enhancements & Roadmap

Phase 2 (Months 3-6):

- ML pipeline for demand forecasting using historical revenue trends
- Customer 360 integration (join streaming orders with loyalty data)
- Real-time anomaly detection for fraud/returns patterns

Phase 3 (Months 6-12):

- Advanced semantic search with LLM fine-tuning on product data
- Graph database integration for product recommendations
- Data mesh implementation for domain-driven ownership

Phase 4 (Year 2):

- Reverse ETL to sync analytics insights back to operational systems
- Multi-region deployment with cross-region replication
- Streaming aggregations with custom aggregation states

SmartGear Retail 2.0 Assignment – Rishabh Srivastava-

Postgraduate Program in Data Science, Data Engineering and AI: Batch 21